



Plataforma de Informes y Reportes

Informe Técnico del Sistema

Empresa

NORTUR (Metrocar) — Transporte / Transfers / Turismo

Versión

1.0 — Junio 2026

Estado

Producción — Desarrollo incremental activo

Base de datos

SQL Server — replicaVPF (réplica de FoxPro productivo)

1. Objetivo del sistema

La plataforma de Informes y Reportes de NORTUR es una aplicación web interna que **reemplaza gradualmente los informes del sistema legacy FoxPro (Metrocar)**. El sistema viejo generaba exportaciones a Excel de forma estática; el nuevo dashboard permite consultas interactivas en tiempo real, con filtros, gráficos y exportación a Excel desde el navegador.

El principio de migración es **reporte por reporte**: se toma un informe del sistema FoxPro, se analiza su lógica de negocio original, y se reconstruye como un panel interactivo moderno — no como una copia, sino como una mejora funcional.

2. Stack tecnológico

Capa	Tecnología	Versión mínima
Lenguaje	Python	3.12
Framework web / dashboard	Streamlit	1.58+
Componente de menú	streamlit-option-menu	0.4+
Gráficos interactivos	Plotly	5.0+
Acceso a base de datos	pyodbc	5.1+
Driver SQL Server	ODBC Driver 18 for SQL Server	—
Exportación a Excel	openpyxl	3.1+
Manipulación de datos	pandas	2.0+

Streamlit actúa internamente como una SPA (Single Page Application) con React + WebSocket: cambiar de reporte o aplicar un filtro no recarga la página completa — actualiza sólo los componentes afectados.

3. Fuente de datos

Base de datos: replicaVPF

Parámetro	Valor
Motor	Microsoft SQL Server (Express)
Servidor	DESKTOP-CV6LF00\SQLEXPRESS
Base de datos	replicaVPF
Driver ODBC	ODBC Driver 18 for SQL Server
Usuario	sa
Total de tablas	108 tablas
Tipo	Réplica de la base productiva FoxPro

replicaVPF es una réplica sincronizada de la base productiva del sistema Metrocar (FoxPro). Contiene los mismos datos que el sistema operativo, pero en formato SQL Server. Esto permite consultar datos en tiempo real sin impactar el sistema de producción FoxPro.

Reglas de calidad de datos

La tabla principal (**viaje**) contiene fechas corruptas (año 309, año 2252, etc.) provenientes del sistema FoxPro original. Para evitar que contaminen los informes, todas las consultas acotan las fechas al rango válido:

- Fecha mínima: 1 de enero de 2021
- Fecha máxima: 31 de diciembre de 2027

4. Estructura de archivos

Archivo / Carpeta	Descripción
app.py	Shell principal: menú, router, CSS global, header, autenticación
auth.py	Login, sesión persistente, logout
config.py	Conexión a SQL Server (un solo lugar)
data.py	Capa de datos: todas las queries SQL centralizadas
tema.py	Paleta de colores corporativa NORTUR
ui.py	Componentes de interfaz reutilizables (selector fechas, pivote)
requirements.txt	Dependencias Python del proyecto
reportes/reservas_fecha_servicio.py	Informe 1: Reservas por fecha y servicio
reportes/reservas_banda_horaria.py	Informe 2: Reservas por banda horaria
reportes/trafico_operacion.py	Informe 3: Tráfico / Operación
.streamlit/config.toml	Tema visual: colores y tipografía de Streamlit
docs/	Documentación técnica del proyecto

5. Descripción de los módulos principales

app.py — Shell principal

Punto de entrada de la aplicación. Realiza tres funciones: (1) Llama a **auth.requerir_login()** antes de cualquier render — si el usuario no está logueado, muestra el formulario y detiene la ejecución. (2) Define el **CSS global** que aplica a todos los reportes (tarjetas, KPIs, header, sidebar, botones). (3) Arma el **menú de navegación** con dos niveles y enruta al módulo elegido.

auth.py — Autenticación

Maneja el login y la sesión. El usuario ingresa nombre y contraseña; se valida contra la tabla **[replicaVPF].[dbo].[usuario]** (password en texto plano, mismo formato que el sistema FoxPro original). Si es

correcto, se guarda **st.session_state.logueado = True** y el nombre del usuario. La sesión persiste mientras la pestaña del navegador esté abierta.

data.py — Capa de datos

Todas las consultas SQL viven en este archivo. Los módulos de reportes nunca escriben SQL directamente — consumen funciones de data.py. Las funciones están decoradas con **@st.cache_data(ttl=300)**: los resultados se cachean 5 minutos para no repetir consultas idénticas.

Función	Descripción
servicios()	Catálogo de tipos de servicio (código + nombre)
reservas_por_fecha_servicio()	Reservas agrupadas por fecha y tipo de servicio
bandas_horarias()	Franjas horarias definidas en tabla viaje_horario
reservas_por_banda_horaria()	Reservas agrupadas por fecha y franja horaria
trafico_por_dimension()	Viajes por cliente, chofer o tipo de vehículo
trafico_evolucion()	Serie diaria de viajes y pax

tema.py — Paleta corporativa

Centraliza todos los colores de la marca NORTUR. Los módulos importan desde acá en lugar de hardcodear valores hexadecimales. Si la empresa cambia su identidad visual, se modifica un solo archivo.

Constante	Color hex	Uso
AZUL	#003AA0	Color principal, KPIs, bordes
AZUL_NOCHE	#112F5B	Header, títulos, texto importante
NARANJA	#F99410	Acento, highlights, logo
AZUL_CLARO	#E8EFF9	Fondos suaves, badges
VERDE	#1E9E6A	Métricas positivas (pax)
ROJO	#D64545	Métricas negativas (cancelados)
PALETA_CUALITATIVA	12 colores	Gráficos con múltiples series
ESCALA_AZUL	Degradado	Gráficos de magnitud continua

ui.py — Componentes reutilizables

selector_fechas(): Dos campos de fecha separados (Desde / Hasta) en una fila. Por defecto: últimos 60 días. Valida que Desde no sea mayor que Hasta. Respeta el rango de fechas válido de config.py.

mostrar_pivote(): Renderiza tablas pivote sin errores de serialización Arrow. Cuando pandas genera un pivot_table con margins=True, la fila 'TOTAL' mezcla fechas (datetime) con texto (str) en el índice — PyArrow falla. Esta función convierte todas las fechas a texto dd/mm/yyyy antes de renderizar.

6. Modelo de datos principal

Tabla viaje — 512.876 filas (una fila = una reserva)

Es la tabla central de todo el sistema. Cada fila representa una reserva de transporte.

Campo	Tipo	Descripción
f_reserva	date	Fecha de la reserva (campo principal de filtros)
f_pedido	date	Fecha en que se cargó el pedido
id_servici	nvarchar(15)	Código de servicio (FK → tabla servicio)
id_cliente	nvarchar(15)	Código de cliente
nombre_cli	nvarchar(50)	Nombre del cliente (desnormalizado)
nombre_cho	nvarchar	Nombre del chofer asignado
id_chofer	nvarchar(15)	Código del chofer
id_vehicul	nvarchar(15)	Código del vehículo asignado
pax	int	Cantidad de pasajeros
estado_via	nvarchar(15)	Estado: FACTURADO / CANCELADO / FINALIZADO...
origen	char(1)	'T' = transportación, 'P' = plantilla
hs_inicio	datetime2	Hora de inicio del servicio
total / importe	decimal	Importes (muchos registros vienen NULL)
_deleted	bit	1 = borrado lógico — siempre filtrar _deleted = 0

Estados de viaje (estado_via):

Estado	Cantidad aprox.
FACTURADO	454.701
FINALIZADO	21.878
CANCELADO	21.287
SIN ASIGNAR	14.943
ASIGNADO	67

Tabla servicio — 61 filas

Catálogo de tipos de servicio. PK: id_servici. Los más frecuentes: CABECERA POR KILOMETRO, CABECERA POR SERVICIO, EZEIZA, TRASLADO, AEROPARQUE, CITY TOUR 4 HS, CENA SHOW, GUARDIA 8 HS.

Tabla viaje_horario — 6 filas (franjas horarias)

Franja	Desde	Hasta
Madrugada 1	00:00	00:01
Madrugada 2	00:02	06:29
Mañana temprana	06:30	08:29
Mañana / Mediodía	08:30	14:00
Tarde	14:01	18:00
Noche	18:01	23:59

Tabla usuario

Campo	Tipo	Descripción
id	int	ID interno del usuario
usuario	nvarchar	Nombre de usuario (ej: ALEJANDRA, SUPERVISOR)
password	nvarchar	Contraseña en texto plano
nivel	nvarchar	Nivel de acceso del sistema FoxPro
acceso	nvarchar	Permisos del sistema FoxPro
_deleted	bit	0 = activo, 1 = eliminado lógicamente

7. Informes disponibles

Informe 1: Reservas por fecha y servicio

Módulo: reportes/reservas_fecha_servicio.py

Origen FoxPro: El código original no estaba en disco — se reconstruyó desde la base de datos con la colaboración del cliente.

Qué muestra:

- KPIs: total reservas, total pax, canceladas (con %), servicios distintos
- Gráfico de barras: evolución diaria por tipo de servicio
- Tarjetas por servicio: reservas / pax / cancelaciones
- Tabla pivote: fecha × servicio con totales
- Gráfico de torta: distribución por servicio (top 12)
- Exportación a Excel (3 hojas: Detalle, Pivote, Ranking)

Filtros:

- Período Desde / Hasta
- Servicios específicos (multiselección)
- Incluir / excluir canceladas (toggle)
- Métrica: Reservas o Pax

Informe 2: Reservas por fecha y banda horaria

Módulo: reportes/reservas_banda_horaria.py

Origen FoxPro: Form trafico_resumen_horario.scx — código disponible en disco. La lógica original fue decodificada y replicada fielmente en el dashboard.

Lógica de negocio original (FoxPro):

- Filtra origen = 'T' (solo servicios de transportación)
- Excluye id_cliente = 'NORTUR' (viajes internos de la empresa)
- Excluye estado_via = 'CANCELADO'
- Asigna cada viaje a una banda horaria según hs_inicio

En el dashboard nuevo, esas reglas son los **valores por defecto** pero el usuario puede modificarlas con toggles (flexibilidad que el FoxPro no tenía).

Qué muestra:

- KPIs: total reservas, total pax, banda pico, bandas con datos
- Gráfico de barras: distribución por franja horaria
- Tarjetas por banda con % del total
- Gráfico de área: evolución temporal por banda
- Tabla pivote: fecha × banda horaria

Validado: enero 2025, reglas FoxPro → 1.336 reservas, banda pico 08:30–14:00 con 547 reservas.

Informe 3: Tráfico / Operación

Módulo: reportes/trafico_operacion.py

Origen FoxPro: Inspirado en trafico_imprime.frx. Vista operativa de viajes agregados por dimensión (cliente, chofer o tipo de vehículo).

Qué muestra:

- KPIs: total viajes, total pax, cantidad de entidades distintas, líder del período
- Ranking horizontal: top N por la dimensión elegida
- Gráfico de área: evolución diaria de viajes/pax
- Tabla con detalle por entidad
- Exportación a Excel (2 hojas: Por dimensión, Evolución diaria)

Filtros:

- Período Desde / Hasta
- Agrupar por: Cliente / Chofer / Tipo de vehículo
- Métrica: Viajes o Pax
- Top N a mostrar (slider 5–30)
- Excluir cliente NORTUR (toggle)
- Incluir canceladas (toggle)

8. Arquitectura del menú

La aplicación **no usa el sistema de páginas nativo de Streamlit** (pages/), porque ese sistema impone su propia navegación en el sidebar y no se combina bien con una barra horizontal custom.

En su lugar se usa el **patrón módulos + router**: cada reporte es un módulo Python con una función render(). El diccionario REPORTES en app.py registra categorías, reportes e íconos. streamlit-option-menu dibuja la barra horizontal con dos niveles (categorías → reportes). El router llama a modulo.render() según la selección.

Para agregar un informe nuevo (3 pasos):

1. Crear reportes/mi_informe.py con def render(): ...
2. Importarlo en app.py
3. Agregarlo al diccionario REPORTES — una línea. El menú se regenera automáticamente.

9. Origen de la lógica de negocio (FoxPro)

El sistema FoxPro completo está en **C:\MetroCarSys**. Contiene: 378 formularios (.scx/.sct), ~40 reportes impresos (.frx), programas (.prg) y menús (.mpr). Los .scx y .frx son tablas DBF de Visual FoxPro con el código fuente almacenado en campos memo — requieren un parser específico para leerlos.

Regla práctica al armar un informe nuevo:

1. Buscar el form/programa en C:\MetroCarSys
2. Si está: leer la lógica original y replicarla (priorizando fidelidad)
3. Si no está en disco: reconstruir desde la base y pedir captura del Excel productivo como referencia

Advertencia: el ejecutable metrocar.exe productivo puede estar más actualizado que el fuente en disco. Verificar siempre caso por caso.

10. Cómo levantar la aplicación

Comando de inicio:

```
cd "c:\Users\HP\OneDrive\CLAUDE CODE\Metrocar Nortur Informes y Reportes"
streamlit run app.py
```

URL de acceso: <http://localhost:8501>

Requisitos previos:

- Python 3.12 instalado
- ODBC Driver 18 for SQL Server instalado
- SQL Server DESKTOP-CV6LF00\SQLEXPRESS corriendo con base replicaVPF accesible
- Dependencias Python: `pip install -r requirements.txt`

11. Próximos informes planificados

Informe	Form FoxPro	Categoría
Viajes detalle	trafico_informe, trafico_imprime	Tráfico
Cuenta Corriente clientes	ctacte_saldo_cliente	Financiero
Liquidación choferes	liquidacion_resumen	Pagos
Combustible	—	Flota
Taller / Service	—	Flota

12. Convenciones y buenas prácticas

- Siempre filtrar `_deleted = 0` en todas las consultas a viaje y tablas relacionadas.
- Acotar fechas al rango válido (`FECHA_MIN_VALIDA` / `FECHA_MAX_VALIDA`) para excluir datos corruptos.
- No escribir SQL en los módulos de reporte — todas las queries van en `data.py`.
- No hardcodear colores hex — importar siempre desde `tema.py`.
- No duplicar widgets — si un componente aparece en 2+ reportes, va a `ui.py`.
- Los campos total/importe tienen muchos NULL — no usar para métricas de facturación sin validar antes. Las métricas confiables hoy son cantidad de reservas y pax.